

PrivateGPT

Um assistente privado com pesquisa sobre documentos locais, empacotado para distribuições Linux modernas

João Miguel Caires Fernandes

Nº 29964 · j.miguel.fernandes@ipvc.pt

IPVC · ESTG · Engenharia Informática

Unidade Curricular: Integração de Sistemas · 2026/2027

Orientadores:

Prof. Jorge Ribeiro e Prof. Rodrigo Rodrigues

September 26, 2026

Estrutura da apresentação

- ① O que é o PrivateGPT
- ② Requisitos do projeto
- ③ Privacidade e autenticação
- ④ O problema da portabilidade
- ⑤ Arquitetura da compilação
- ⑥ Empacotamento como Appliance
- ⑦ Instalação com um só comando
- ⑧ Verificação e resultados
- ⑨ Demonstração
- ⑩ Considerações finais e bibliografia

O que é o PrivateGPT

Em poucas palavras

Um assistente de linguagem que procura responder a perguntas com base nos **documentos do próprio utilizador** — sem que esses documentos, ou as conversas daí resultantes, saiam da máquina.

O que se pretende

- Ingerir documentos locais (PDF, texto, etc.)
- Indexar o conteúdo para pesquisa semântica
- Responder com base nos excertos encontrados
- Manter cada conversa em ambiente local

O que se procura evitar

- Enviar documentos para serviços externos
- Guardar histórico em servidores remotos
- Expor conversas em texto simples no disco

A privacidade procura ser garantida desde a conceção, e não apenas como política.

Requisitos propostos

Requisitos do trabalho

- Desenvolver um assistente que responda sobre documentos locais
- **Um único comando** para instalar e executar
- **Portabilidade** entre distribuições Linux de 64 bits
- Plataforma alvo: Linux (x86_64)

A tensão identificada

A facilidade de utilização e a portabilidade puxam em sentidos opostos. Um binário compilado num sistema recente é simples de produzir, mas corre em pouquíssimos sistemas.

Proteção das conversas

O problema

Se as conversas ficam guardadas em texto simples no disco, qualquer pessoa com acesso ao computador as pode ler. A aplicação procura evitar isso.

Cifra das conversas

- Cada conversa é guardada **cifrada**
- A chave é derivada da palavra-passe do utilizador
- Sem a palavra-passe correta, o conteúdo é ilegível

Autenticação por *hash*

- A palavra-passe **não é guardada** em texto simples
- Guarda-se apenas um *hash* resultante de uma função de derivação
- O *hash* permite verificar a autenticidade de quem se identifica

A palavra-passe nunca é escrita em disco. Só o resultado da sua transformação é guardado.

Porque é que os binários falham: a glibc

- Praticamente todos os binários dinâmicos de Linux dependem da **glibc**
- Um binário requer, no mínimo, a versão de glibc contra a qual foi compilado
- Compilar num sistema recente implica que o binário só corra em sistemas igualmente recentes

Sistema	glibc	Corre binário compilado no anfitrião?
Kali 2026.3 (máquina de desenvolvimento)	2.43	sim
Kali rolling 2025	2.41	não
Ubuntu 24.04 LTS	2.39	não
Debian 12 Bookworm	2.36	não
Ubuntu 22.04 LTS	2.35	não
Objetivo mínimo	2.31	—

Uma observação relevante

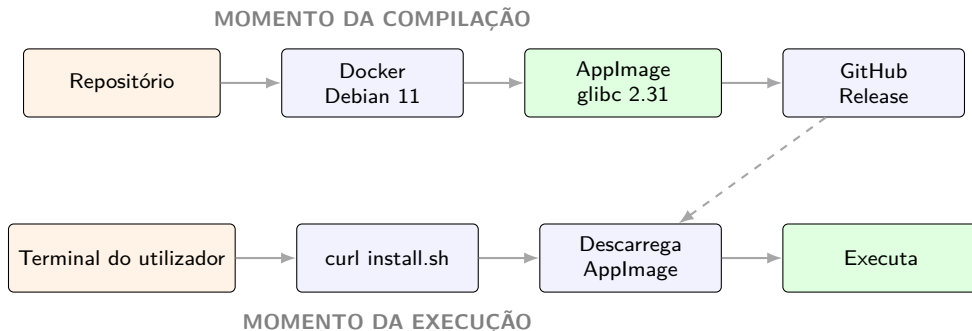
A portabilidade é uma característica do
momento da compilação,
e não do **momento da execução.**

- Compilar dentro de um contentor *não* torna, por si só, um binário mais portátil
- Apenas muda *onde* a compilação corre
- O que importa é **contra que versão de glibc o binário foi compilado**

A abordagem adotada

Fixar o ambiente de compilação a uma base **antiga**. O resultado passa, então, a correr em tudo o que seja mais recente.

Duas etapas, dois públicos



O Docker é uma ferramenta de **compilação**. O utilizador final não precisa de o instalar.

Porquê python:3.11-bullseye?

- **Debian 11** ⇒ glibc **2.31**
- Cobre qualquer distribuição de 64 bits a partir de **2021**
- Imagem **completa** (não -slim): inclui o tkinter
- Python 3.11 já instalado — sem PPA nem compilação a partir do código-fonte

Porque não compilar no Kali?

O sistema anfitrião corre glibc **2.43**.
Uma compilação local exigiria 2.43 — e correria em muito poucos sistemas.

Base de compilação	glibc mínima exigida	
CentOS 7	2.17	(2014+)
Debian 10	2.28	(2018+)
Debian 11	2.31	(2021+)
Debian 12	2.36	(2023+)

João Miguel Caires Fernandes · N.º 29964 ·

Do código Python ao executável

O problema

Um programa Python precisa de um interpretador, de bibliotecas e de um ambiente configurado para correr. Pedir isso a quem quer apenas usar a aplicação não é razoável.

O que o PyInstaller faz

- Analisa o código e descobre as dependências
- Reúne o interpretador Python e as bibliotecas
- Produz um executável autónomo, sem instalação

O que fica de fora

- Não resolve o problema da glibc
- Não empacota dados externos por omissão
- Não produz, sozinho, um formato distribuível

O PyInstaller resolve a dependência do Python. O Applmage resolve a dependência do sistema. São passos complementares.

A verificar a portabilidade

Extrair o AppImage e inspecionar a biblioteca Tcl/Tk incluída:

```
./dist/PrivateGPT-x86_64.AppImage --appimage-extract  
strings squashfs-root/usr/bin/_internal/libtcl8.6.so \  
    | grep -o 'GLIBC_2\.[0-9]*' | sort -Vu | tail -1
```

Resultado esperado

GLIBC_2.31

Compilado no contentor.

Corre em qualquer Linux de 64 bits a partir de 2021.

Resultado indesejado

GLIBC_2.43

A compilação usou a glibc do anfitrião.

O contentor foi contornado.

Publicação no GitHub Releases

- 1 Executar `./build.sh` $\Rightarrow$ `dist/PrivateGPT-x86_64.AppImage`
- 2 Criar uma *Release* no GitHub, com uma etiqueta (por exemplo, `v0.1.0`)
- 3 Carregar o AppImage como recurso da *release*, com o nome exatamente igual ao esperado pelo instalador

O alias de URL que dispensa indicar a versão

```
https://github.com/UTILIZADOR/REPO/releases/  
latest/download/PrivateGPT-x86_64.AppImage
```

- `latest` aponta sempre para a versão estável mais recente
- Não é necessário editar o script ao publicar uma nova versão

A experiência com um só comando

```
curl -fsSL <URL do install.sh> | bash
```



Tratado automaticamente

- Verificação da plataforma e arquitetura
- Criação da pasta de instalação
- Configuração do PATH
- Detecção da libfuse2
- Execução

Fixar uma versão (opcional)

```
PRIVATEGPT_VERSION=v0.1.0 curl ... | bash
```

Descarrega essa versão específica em vez de latest.

Matriz de compatibilidade

Distribuição	Lançamento	glibc	Estado
Ubuntu 20.04 LTS	2020	2.31	compatível
Debian 11 Bullseye	2021	2.31	compatível
Kali 2021.x	2021	2.31	compatível
Fedora 34+	2021	2.33	compatível
Ubuntu 22.04 LTS	2022	2.35	compatível
Debian 12 Bookworm	2023	2.36	compatível
Ubuntu 24.04 LTS	2024	2.39	compatível
Kali rolling	2025	2.41	compatível

Mínimo verificado: GLIBC_2.31. Qualquer Linux de 64 bits a partir de 2021.

O que foi possível alcançar

Facilidade de utilização

- **Um único comando** para instalar e executar
- Sem Docker, sem Python, sem passos manuais para o utilizador
- Cerca de 5 segundos entre o comando e a aplicação em execução

Portabilidade e privacidade

- Espera-se que corra em **qualquer Linux de 64 bits a partir de 2021**
- Um único artefacto autónomo
- Conversas cifradas e palavra-passe protegida por *hash*

Privado desde a conceção, portátil desde a compilação, instalável com uma única linha.

Demonstração

Todos os três GPTs disponíveis em:

`https://seaman6969.github.io/
GPT_unipvc_estg_ei_is_2026_2027/`

A página de entrada reúne o LocalGPT, o PrivateGPT e o WebGPT, cada um com o seu comando de instalação pronto a copiar.

```
$ curl -fsSL https://raw.githubusercontent.com/Seaman6969/  
PrivateGPT_unipvc_estg_ei_is_2026_2027/main/install.sh | bash
```

(passagem para o terminal para demonstração ao vivo)

Algumas ilações

- ❶ **A portabilidade decide-se na compilação.** Colocar o *utilizador* dentro de um contentor acrescenta complexidade sem benefício.
- ❷ **A base mais antiga compensa.** O Debian 11 fixa um mínimo de 2.31 que cobre vários anos de distribuições.
- ❸ **Um artefacto vale mais do que muitos passos.** O Applmage junta instalação, dependências e execução num único ficheiro.
- ❹ **A privacidade exige medidas concretas.** Cifrar as conversas e derivar a palavra-passe protege os dados mesmo em caso de acesso ao disco.

Trabalho futuro

- Estender o mínimo de glibc a distribuições de 2014 através de uma base CentOS 7
- Automatizar compilações e publicações com GitHub Actions a cada etiqueta
- Assinar digitalmente os Applimages para verificação de integridade
- Avaliar **Applimages estáticos de Tipo 3** para eliminar a dependência da libfuse2
- Explorar esquemas de recuperação de chave em caso de esquecimento da palavra-passe

Bibliografia

Modelos e execução local

- DeepSeek — <https://www.deepseek.com>
- Ollama — <https://ollama.com>
- llama-cpp-python — <https://github.com/abetlen/llama-cpp-python>
- Hugging Face Hub — <https://huggingface.co>

Bibliotecas Python

- Transformers — <https://huggingface.co/docs/transformers>
- LangChain — <https://python.langchain.com>
- ChromaDB — <https://www.trychroma.com>
- sentence-transformers — <https://www.sbert.net>

Empacotamento e distribuição

- PyInstaller — <https://pyinstaller.org>
- AppImage — <https://appimage.org>
- Docker — <https://docs.docker.com>
- GitHub Pages — <https://pages.github.com>

Projeto

- Repositório PrivateGPT — https://github.com/Seaman6969/PrivateGPT_unipvc_estg_ei_is_2026_2027
- Repositório LocalGPT — https://github.com/Seaman6969/LocalGPT_unipvc_estg_ei_is_2026_2027
- Repositório WebGPT — https://github.com/Seaman6969/WebGPT_unipvc_estg_ei_is_2026_2027